Jest: How to mock one specific method of a class

Asked 3 years, 3 months ago Active 1 month ago Viewed 81k times

101

27

Let's suppose I have the following class:

```
export default class Person {
    constructor(first, last) {
        this.first = first;
        this.last = last;
    }
    sayMyName() {
        console.log(this.first + " " + this.last);
    }
    bla() {
        return "bla";
    }
}
```

Suppose I want to create a mocked class where method 'sayMyName' will be mocked and method 'bla' will stay as is.

The test I wrote is:

```
const Person = require("../Person");

jest.mock('../Person', () => {
    return jest.fn().mockImplementation(() => {
        return {sayMyName: () => {
            return 'Hello'
        }};
    });
});


let person = new Person();
test('MyTest', () => {
    expect(person.sayMyName()).toBe("Hello");
    expect(person.bla()).toBe("bla");
})
```

The first 'expect' statement passes, which means that 'sayMyName' was mocked successfully. But, the second 'expect' fails with the error:

> TypeError: person.bla is not a function

I understand that the mocked class erased all methods. I want to know how to mock a class such that only specific method(s) will be mocked.

javascript jestjs

Share Follow

edited Sep 24 '20 at 2:58
risingTide
1,396 ● 7 ● 22 ● 50

asked Apr 29 '18 at 21:21
CrazySynthax
9,902 ● 23 ● 72 ● 138

Please change your selected answer, the correct answer should be @blade's. – peja Apr 27 '20 at 13:13

7 Answers

Active | Oldest | Votes

31

Edit 05/03/2021

I see a number of people disagree with the below approach, and that's cool. I do have a slight disagreement with @blade's approach, though, in that it actually doesn't test the class because it's using `mockImplementation`. If the class changes, the tests will still always pass giving false positives. So here's an example with `spyOn`.

```
// person.js
export default class Person {
  constructor(first, last) {
      this.first = first;
      this.last = last;
  }
  sayMyName() {
      return this.first + " " + this.last; // Adjusted to return a value
  }
  bla() {
      return "bla";
  }
}
```

and the test:

```
import Person from './'

describe('Person class', () => {
  const person = new Person('Guy', 'Smiley')

  // Spying on the actual methods of the Person class
  jest.spyOn(person, 'sayMyName')
  jest.spyOn(person, 'bla')

  it('should return out the first and last name', () => {
    expect(person.sayMyName()).toEqual('Guy Smiley') // deterministic
    expect(person.sayMyName).toHaveBeenCalledTimes(1)
  });
  it('should return bla when blah is called', () => {
    expect(person.bla()).toEqual('bla')
    expect(person.bla).toHaveBeenCalledTimes(1)
  })
});
```

Cheers!

I don't see how the mocked implementation actually solves anything for you. I think this makes a bit more sense

```
import Person from "./Person";

describe("Person", () => {
  it("should...", () => {
    const sayMyName = Person.prototype.sayMyName = jest.fn();
    const person = new Person('guy', 'smiley');
    const expected = {
      first: 'guy',
      last: 'smiley'
    }

    person.sayMyName();

    expect(sayMyName).toHaveBeenCalledTimes(1);
    expect(person).toEqual(expected);
  });
});
```

Share Follow

edited May 3 at 16:55

answered Apr 30 '18 at 21:19

sesamechicken

1,565 ● 11 ● 18

13 I don't know the answer to this, so genuinely curious: would this leave the Person.prototype.sayMyName altered for any other tests running after this one? – Martin Oct 8 '18 at 18:52

7 @Martin Yes, it does. – Frondor Oct 15 '18 at 18:44

11 I don't think this is a good practice. It's not using Jest or any other framework to mock the method and you'll need extra effort to restore the method. – Bruno Brant Sep 4 '19 at 15:16

5 See stackoverflow.com/a/56565849/1248209 for answer on how to do this properly in Jest using spyOn. – Lockyy Oct 21 '19 at 14:06

288

Using `jest.spyOn()` is the proper *Jest* way of mocking a single method and leaving the rest be. Actually there are two slightly different approaches to this.

1. Modify the method only in a single object

```
import Person from "./Person";

test('Modify only instance', () => {
    let person = new Person('Lorem', 'Ipsum');
    let spy = jest.spyOn(person, 'sayMyName').mockImplementation(() => 'Hello');

    expect(person.sayMyName()).toBe("Hello");
    expect(person.bla()).toBe("bla");
```

```
    // unnecessary in this case, putting it here just to illustrate how to "unmock" a
method
    spy.mockRestore();
});
```

2. Modify the class itself, so that all the instances are affected

```
import Person from "./Person";

beforeAll(() => {
    jest.spyOn(Person.prototype, 'sayMyName').mockImplementation(() => 'Hello');
});

afterAll(() => {
    jest.restoreAllMocks();
});

test('Modify class', () => {
    let person = new Person('Lorem', 'Ipsum');
    expect(person.sayMyName()).toBe("Hello");
    expect(person.bla()).toBe("bla");
});
```

And for the sake of completeness, this is how you'd mock a static method:

```
jest.spyOn(Person, 'myStaticMethod').mockImplementation(() => 'blah');
```

Share Follow

answered Jun 12 '19 at 15:36

blade

8,807 ● 6 ● 32 ● 35

12 It's a shame the official documentation doesn't say this. The official documentation is an incomprehensible mess of module factories, class mocks hand rolled using object literals, stuff stored in special directories, and special case processing based on property names. jestjs.io/docs/en/... – Neutrino Jan 24 at 10:56

I've edited my answer above. @blade Using `mockImplementation` will always yield a passing test and if the class changes, a false positive. – sesamechicken May 3 at 16:56

2 @sesamechicken Not sure I follow with the always passing test. You should never test a mocked method directly, that would make zero sense. A usual use case is to mock a method that provides data for a different method that you're actually testing. – blade May 5 at 12:07

>You should never test a mocked method directly That's exactly what the example above demonstrates using `mockImplementation`. – sesamechicken May 6 at 13:54

It does, sure, but it answers the OP's question. I think it would be off-topic to go into details of correct testing in this answer and would detract from the point. – blade May 7 at 16:04

9

Have been asking similar question and I think figured out a solution. This should work no matter where Person class instance is actually used.

```
const Person = require("../Person");

jest.mock("../Person", function () {
    const { default: mockRealPerson } = jest.requireActual('../Person');

    mockRealPerson.prototype.sayMyName = function () {
        return "Hello";
    }

    return mockRealPerson
});

test('MyTest', () => {
    const person = new Person();
    expect(person.sayMyName()).toBe("Hello");
    expect(person.bla()).toBe("bla");
});
```

Share Follow

edited Jan 7 '19 at 11:09

answered Jan 6 '19 at 0:08

madhamster
111 ● 1 ● 4

7

If you are using Typescript, you can do the following:

```
Person.prototype.sayMyName = jest.fn().mockImplementationOnce(async () =>
        await 'my name is dev'
);
```

And in your test, you can do something like this:

```
const person = new Person();
const res = await person.sayMyName();
expect(res).toEqual('my name is dev');
```

Hope this helps someone!

Share Follow

answered Oct 18 '18 at 2:48

Saif Asad
573 ● 7 ● 6

How do we assert that the mock was called? – Jasper Blues Feb 2 '19 at 4:30

6

rather than mocking the class you could extend it like this:

```
class MockedPerson extends Person {
  sayMyName () {
    return 'Hello'
  }
}
```

```
// and then
let person = new MockedPerson();
```

Share Follow

answered Apr 30 '18 at 14:59

Billy Reilly
932 ● 7 ● 9

Brilliant idea! I find it very much useful for mocking classes! ♡ – A.N.M. Saiful Islam Oct 9 '20 at 13:58

5

Not really answer the question, but I want to show a use case where you want to mock a dependent class to verify another class.

For example: `Foo` depends on `Bar`. Internally `Foo` created an instance of `Bar`. You want to mock `Bar` for testing `Foo`.

Bar class

```
class Bar {
  public runBar(): string {
    return 'Real bar';
  }
}

export default Bar;
```

Foo class

```
import Bar from './Bar';

class Foo {
  private bar: Bar;

  constructor() {
    this.bar = new Bar();
  }

  public runFoo(): string {
    return 'real foo : ' + this.bar.runBar();
  }
}

export default Foo;
```

The test:

```
import Foo from './Foo';
import Bar from './Bar';

jest.mock('./Bar');
```

```
describe('Foo', () => {
  it('should return correct foo', () => {
    // As Bar is already mocked,
    // we just need to cast it to jest.Mock (for TypeScript) and mock whatever you want
    (Bar.prototype.runBar as jest.Mock).mockReturnValue('Mocked bar');
    const foo = new Foo();
    expect(foo.runFoo()).toBe('real foo : Mocked bar');
  });
});
```

Note: this will not work if you use arrow functions to define methods in your class (as they are difference between instances). Converting it to regular instance method would make it work.

See also jest.requireActual(moduleName)

Share Follow

edited Jul 1 at 15:09

answered Oct 25 '20 at 17:14

Ninh Pham

4,604 ● 37 ● 26

1

I've combined both @sesamechicken and @Billy Reilly answers to create a util function that mock (one or more) specific methods of a class, without definitely impacting the class itself.

```
/**
* @CrazySynthax class, a tiny bit updated to be able to easily test the mock.
*/
class Person {
    constructor(first, last) {
        this.first = first;
        this.last = last;
    }

    sayMyName() {
        return this.first + " " + this.last + this.yourGodDamnRight();
    }

    yourGodDamnRight() {
        return ", you're god damn right";
    }
}

/**
 * Return a new class, with some specific methods mocked.
 *
 * We have to create a new class in order to avoid altering the prototype of the class
itself, which would
 * most likely impact other tests.
 *
 * @param Klass: The class to mock
 * @param functionNames: A string or a list of functions names to mock.
 * @returns {Class} a new class.
 */
export function mockSpecificMethods(Klass, functionNames) {
    if (!Array.isArray(functionNames))
        functionNames = [functionNames];
```

```
    class MockedKlass extends Klass {
    }

    const functionNamesLenght = functionNames.length;
    for (let index = 0; index < functionNamesLenght; ++index) {
        let name = functionNames[index];
        MockedKlass.prototype[name] = jest.fn();
    };

    return MockedKlass;
}

/**
* Making sure it works
*/
describe('Specific Mocked function', () => {
    it('mocking sayMyName', () => {
        const walter = new (mockSpecificMethods(Person, 'yourGodDamnRight'))('walter',
'white');

        walter.yourGodDamnRight.mockReturnValue(", that's correct"); //
yourGodDamnRight is now a classic jest mock;

        expect(walter.sayMyName()).toBe("walter white, that's correct");
        expect(walter.yourGodDamnRight.mock.calls.length).toBe(1);

        // assert that Person is not impacted.
        const saul = new Person('saul', 'goodman');
        expect(saul.sayMyName()).toBe("saul goodman, you're god damn right");
    });
});
```

Share Follow

edited Dec 15 '18 at 18:27

answered Dec 15 '18 at 18:22

jolancornevin

741 ● 8 ● 7